

CAN BUS COMMUNICATION

Complete Technical Reference & Study Notes

- Controller Area Network (CAN) Protocol
- Standard (2.0A) vs Extended (2.0B) Frames
 - DBC File Format & Signal Decoding
- Termination – Theory, Logic & Multi-Node Rules
- Physical Layer, Error Handling & Best Practices

ISO 11898 | SAE J1939 | ISO 15765

1. Introduction to CAN Bus

Controller Area Network (CAN) is a robust, serial communication protocol originally developed by Robert Bosch GmbH in 1983 and standardized under ISO 11898. It was designed to allow microcontrollers and devices to communicate with each other without a host computer.

CAN is a multi-master, message-based protocol — meaning any node can transmit when the bus is idle, and arbitration is built-in. It is widely used in automotive, industrial automation, medical devices, and aerospace systems.

1.1 Why CAN?

- Multi-master: Any device can initiate communication
- Prioritized arbitration: Lower CAN-ID = higher priority (no collision)
- Error detection & correction built into hardware
- Differential signaling (CAN_H and CAN_L) = excellent noise immunity
- Only 2 wires needed for all nodes on the network
- Bus lengths from a few meters to 5+ km at low baud rates

1.2 CAN Physical Signals: CAN_H and CAN_L

CAN uses differential signaling across two wires: CAN_H (High) and CAN_L (Low). This makes it immune to common-mode noise.

Bus State	CAN_H Voltage	CAN_L Voltage	Differential
Recessive (Logic 1)	~2.5V	~2.5V	~0V
Dominant (Logic 0)	3.5V (2.75–4.5V)	1.5V (0.5–2.25V)	~2V



The DOMINANT state (logic 0) always wins over RECESSIVE (logic 1). This is the key to CAN's non-destructive arbitration — the node with the lowest CAN ID (most dominant bits) wins the bus.

2. Standard CAN Frame (2.0A) — 11-bit ID

The Standard CAN frame (defined in CAN 2.0A / ISO 11898) uses an 11-bit Message Identifier, allowing up to 2,048 unique message IDs. This is sufficient for smaller networks with fewer nodes and message types.

2.1 Standard Frame Structure

Standard CAN Data Frame layout (read left to right, transmitted MSB first):

SOF	ID (11-bit)	RTR	IDE IDE=0	r0	DLC	Data (0–8 bytes)	CRC (15- bit)	CRC Del	ACK Slot	ACK Del	EOF
1 bit	11 bits	1 bit	1 bit	1 bit	4 bits	0–64 bits	15 bits	1	1	1	7 bits

2.2 Field-by-Field Explanation

Field	Bits	Description
SOF	1	Start of Frame — a single dominant bit; synchronizes all nodes to frame start.
Identifier (ID)	11	Message priority and content type. Lower value = higher priority. Transmitted MSB first.
RTR	1	Remote Transmission Request — 0=data frame, 1=remote frame (requesting data).
IDE	1	Identifier Extension — 0 (dominant) means Standard frame. If 1 (recessive), it's Extended.
r0	1	Reserved bit — always 0 (dominant). Receivers ignore it; future use by protocol.
DLC	4	Data Length Code — number of data bytes in the payload (0 to 8 bytes).
Data Field	0–64	Actual payload — 0 to 8 bytes. Content defined by the application (signal encoding, DBC, etc.).
CRC	15	Cyclic Redundancy Check — polynomial division of all preceding bits for error detection.
CRC Delimiter	1	Recessive bit — separates CRC from ACK slot.
ACK Slot	1	Transmitter sends recessive; any receiver that got correct frame sends dominant here (ACK).
ACK Delimiter	1	Recessive bit after ACK slot.
EOF	7	End of Frame — 7 recessive bits indicating frame end.

3. Extended CAN Frame (2.0B) — 29-bit ID

Extended CAN frame (CAN 2.0B / ISO 11898) uses a 29-bit Identifier = 11-bit Base ID + 18-bit Extension ID. This supports over 536 million unique IDs, essential for complex vehicle networks (e.g., trucks, J1939, heavy machinery).

3.1 Extended Frame Structure

Extended CAN Data Frame layout (the arbitration field is longer compared to Standard):

SOF	Base ID (11-bit)	SRR	IDE IDE=1	Ext ID (18-bit)	RTR	r1,r0	DLC	Data (0–8 bytes)	CRC	ACK	EOF
1	11 bits	1	1	18 bits	1	2	4 bits	0–64 bits	15	2	7

3.2 Extended Frame — New Fields

Field	Explanation
SRR (1 bit)	Substitute Remote Request — always recessive (1). Appears where RTR is in Standard frames. Acts as a placeholder to maintain backward compatibility during arbitration. It is NOT the RTR bit.
IDE (1 bit)	Identifier Extension — recessive (1) in Extended frames. This tells all nodes the frame is Extended. In Standard frames IDE=0 (dominant). This is how the bus knows which format is being used.
Ext ID (18 bits)	18-bit extension to the base 11-bit ID. Combined with the 11-bit base, creates a 29-bit identifier. Transmitted MSB first.
RTR (1 bit)	Remote Transmission Request in the extended arbitration field — same meaning as in Standard: 0=data, 1=remote request.
r1, r0 (2 bits)	Two reserved bits (always dominant/0). In Standard only r0 was present; Extended has r1 and r0 for future protocol extensions.

3.3 How IDE Bit Distinguishes Frames (Critical!)

When two nodes simultaneously transmit — one Standard, one Extended — arbitration determines who wins:

- Standard frame: 11-bit ID → IDE=0 (dominant) is transmitted at bit position 12
- Extended frame: 11-bit base ID → SRR=1 (recessive) → IDE=1 (recessive) then 18-bit ext ID
- If Standard and Extended share same 11-bit base ID, Standard WINS because IDE=0 is dominant
- This ensures backward compatibility between 2.0A and 2.0B nodes on the same bus



KEY RULE: Standard frame always wins over an Extended frame with the same base ID, because IDE=0 (dominant) beats IDE=1 (recessive) during arbitration. Design your ID assignments with this in mind.

4. Standard vs Extended — Full Comparison

Feature	Standard CAN (2.0A)	Extended CAN (2.0B)
ID Length	11 bits	29 bits (11 + 18)
Max Unique IDs	2,048 (2^{11})	536,870,912 (2^{29})
IDE Bit Value	0 (dominant)	1 (recessive)
Frame Size (bits)	~47 + data	~67 + data (longer header)
SRR Bit	Not present	Substitute Remote Request
Arbitration Field	11-bit ID + RTR	11-bit + SRR + IDE + 18-bit + RTR
DLC field	4 bits (0–8 bytes)	4 bits (0–8 bytes)
Max Data	8 bytes	8 bytes (CAN FD: 64 bytes)
Backward Compatible	With Extended	With Standard
Use Case	Simple networks, older ECUs	Large vehicle networks, many nodes
ISO Standard	ISO 11898-1 CAN 2.0A	ISO 11898-1 CAN 2.0B

4.1 When to Use Which?

Use Standard (11-bit) When...	Use Extended (29-bit) When...
Network has fewer than 2,048 message types	Network has many ECUs and complex message structure
Simpler car/motorcycle applications	J1939 (trucks), CANopen, ISOBUS, heavy-duty vehicles
Legacy ECUs or OBD-II (11-bit mode)	Industrial systems with hundreds of sensors/actuators
Lower overhead, slightly faster for same payload	Need globally unique IDs across multiple systems

5. DBC File Format — Database CAN

A DBC (Database CAN) file is a text-based description file that defines all messages, signals, nodes, and their properties on a CAN network. It is the standard format used by tools like Vector CANdb++, CANalyzer, PEAK, and many automotive tools.



DBC files are NOT transmitted over the CAN bus. They are engineering description files used by software tools to interpret and decode raw CAN frames into human-readable signal values.

5.1 DBC File Structure — Keywords

DBC Keyword	Purpose	Example
VERSION	DBC file version string	VERSION ""
NS_	New symbols (namespace list)	NS_ : ...
BS_	Baudrate section	BS_ : 500000
BU_	Network nodes list	BU_ : ECU1 TCU BCM
BO_	Message definition	BO_ 256 EngineData: 8 ECU1
SG_	Signal definition inside message	SG_ RPM : 0 16@1+ (0.1,0) [0 6553.5]
CM_	Comments (for message/signal)	CM_ SG_ 256 RPM "Engine speed";
BA_DEF_	Attribute definition	BA_DEF_ SG_ "InitialValue" FLOAT 0 100;
BA_	Attribute value assignment	BA_ "BusType" "CAN";
VAL_	Value definitions (enums)	VAL_ 256 GearPos 0 "Park" 1 "Drive";
SIG_GROUP_	Signal grouping (for multiplexed)	SIG_GROUP_ 256 Grp1 1 : RPM Torque;

5.2 Sample DBC File

Below is a complete minimal DBC example with two messages and multiple signals:

```
VERSION ""

NS_ :

BS_ : 500000

BU_ : ECU1 TCU BCM DIAG

// ——— Message 1: Engine Data (Standard, 8 bytes) ———
BO_ 256 EngineData: 8 ECU1
  SG_ EngineRPM : 0|16@1+ (0.25,0) [0|16383.75] "rpm" TCU,BCM
  SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|215] "degC" BCM
  SG_ ThrottlePos : 24|8@1+ (0.39216,0) [0|100] "%" TCU
  SG_ EngineLoad : 32|8@1+ (0.39216,0) [0|100] "%" TCU
```

```
// — Message 2: Gear Status (Extended ID = 0x18FF1234) —
BO_ 2550600244 GearStatus: 4 TCU
SG_ CurrentGear : 0|4@1+ (1,0) [0|15] "" BCM
SG_ TargetGear : 4|4@1+ (1,0) [0|15] "" BCM
SG_ GearMode : 8|2@1+ (1,0) [0|3] "" BCM

CM_ SG_ 256 EngineRPM "Engine speed in revolutions per minute";
CM_ SG_ 256 CoolantTemp "Engine coolant temperature";

BA_DEF_ SG_ "SystemSignalLongSymbol" STRING;
BA_DEF_ SG_ "InitialValue" FLOAT 0 100;

VAL_ 2550600244 GearMode 0 "Park" 1 "Reverse" 2 "Neutral" 3 "Drive";
```

5.3 Signal Definition Breakdown (SG_ Line)

Every signal line in a DBC follows a strict format. Here is the anatomy of one signal definition:

```
SG_ EngineRPM : 0|16@1+ (0.25,0) [0|16383.75] "rpm" TCU,BCM
```

SG_ Field	Meaning	Example Value
Signal Name	Unique identifier	EngineRPM
Start Bit	LSB bit position in frame	0
Bit Length	Number of bits	16
Byte Order	1=Intel (little-endian), 0=Motorola (big-endian)	1 (Intel)
Value Type	+ = unsigned, - = signed	+
Factor	Scale: Raw × Factor + Offset = Physical	0.25
Offset	Added after factor multiplication	0
Min/Max	Physical value range	[0 16383.75]
Unit	Physical unit label	rpm
Receiver	Node that reads this signal	TCU,BCM

5.4 Physical Value Calculation

To convert raw CAN data bytes to engineering/physical values:

Physical Value = (Raw Value × Factor) + Offset

Example: Raw=2000, Factor=0.25, Offset=0 → Physical = 2000 × 0.25 + 0 = 500 rpm

6. CAN Bus Termination — Theory & Logic

Termination is one of the most critical aspects of CAN bus physical design. Incorrect termination leads to signal reflections, communication errors, and intermittent failures that are hard to debug.

6.1 Why is Termination Needed?

When a signal (voltage wave) travels along a wire (transmission line) and reaches the end, if there is no load matching the cable's characteristic impedance, the wave reflects back. This causes:

- Signal overshoot and undershoot (voltage spikes and dips)
- Ringing (oscillation at the end of each bit)
- Bit errors — receiver samples wrong voltage during ringing
- The higher the baud rate, the worse the reflection effects



A standard twisted-pair CAN cable has a characteristic impedance of approximately 120Ω. Termination resistors match this impedance to absorb signal energy at the ends, eliminating reflections.

6.2 Standard Termination Topology

The correct CAN bus topology is a straight line (daisy-chain), NOT a star. One 120Ω resistor is placed at EACH PHYSICAL END of the bus:



The two 120Ω resistors in parallel give a total bus impedance of 60Ω — this is the standard CAN bus load impedance that transceivers are designed to drive.

Calculation	Formula	Result
Two 120Ω in parallel	$(120 \times 120) / (120 + 120)$	= 60Ω ✓
One 120Ω only	120Ω (open at other end)	120Ω — reflections!
Three 120Ω in parallel	$120/3 = 40\Omega$	= 40Ω — overload!

6.3 Split Termination

An alternative to the single 120Ω termination is Split Termination — two 60Ω resistors in series with a capacitor (typically 4.7nF to 100nF) from the midpoint to ground:

- Effective impedance is still 60Ω (60+60 = 120Ω equivalent per end, parallel = 60Ω)
- The capacitor filters high-frequency common-mode noise (EMI)
- Recommended for noisy environments (engine bay, motor drives, inverters)
- Used in ISO 11898-2 and CAN FD networks

Standard Termination	Split Termination
One 120Ω resistor at each end	Two 60Ω resistors in series at each end
Simple, cheap	Better EMI rejection
Fine for clean environments	Capacitor to GND at midpoint
Widely used in automotive	Preferred for CAN FD & harsh environments

7. Multi-Device Termination Logic

This is where engineers often make mistakes. The GOLDEN RULE is simple but must be applied correctly regardless of node count:

THE GOLDEN RULE OF CAN TERMINATION

ALWAYS place exactly TWO 120Ω resistors — one at EACH PHYSICAL END of the bus wire.

The number of nodes in between does NOT change this rule.

7.1 Multi-Node Termination Guide

# Nodes	Termination Points	Where to Place	Effective Load
2	2 resistors	One at each node end	60Ω
3	2 resistors	End nodes only (middle node: NO resistor)	60Ω
5	2 resistors	First and last node only	60Ω
10	2 resistors	Physical ends of the bus cable	60Ω
20+	2 resistors (split OK)	Bus endpoints, consider split termination	60Ω

7.2 Scenario Examples — 5 Nodes

Example: 5 ECUs connected on a CAN bus — ECU1, ECU2, ECU3, ECU4, ECU5

✓ CORRECT Wiring:

[120Ω] — ECU1 — ECU2 — ECU3 — ECU4 — ECU5 — [120Ω]

Only ECU1 and ECU5 have 120Ω termination. ECU2, ECU3, ECU4 have NO termination resistors.

✗ WRONG — Every Node Terminated:

[120Ω]—ECU1 [120Ω]—ECU2 [120Ω]—ECU3 [120Ω]—ECU4 [120Ω]—ECU5 [120Ω]

6 resistors × 120Ω = 20Ω effective load. Bus heavily overloaded. Transceivers overheat. Communication fails.

7.3 Stub Lines (T-Branch) — Avoid!

A stub is a branch off the main bus line to reach a node. Stubs cause reflections because they present an impedance discontinuity. Guidelines:

- Keep stubs as short as possible (ideally < 30 cm at 1 Mbps)
- At 500 kbps, stubs should be < 60 cm; at 250 kbps, < 1.2 m

- Never add termination resistors to the end of stubs
- Star topologies (central hub with spokes) are NOT recommended for CAN without special equipment



RULE: Max stub length $\approx$ (Bus length / baud rate) $\times$ constant. In practice, for 1 Mbps CAN, keep stubs under 30 cm. For CAN FD at 5 Mbps, keep stubs under 5 cm.

7.4 All Termination Scenarios Summary

Scenario	Termination	Result	Status
Both ends terminated (correct)	$120\Omega + 120\Omega = 60\Omega$	Signal integrity OK, reflections suppressed	✓ CORRECT
Only one end terminated	120Ω (one end open)	Reflections occur, signal corruption at high baud	⚠ WARNING
No termination	None (bus open)	Severe reflections, communication failure	✗ FAIL
Both ends + middle node	$120\Omega + 120\Omega + \text{extra}$	Total resistance too low ($<60\Omega$), bus overload	✗ FAIL
Split termination ($2 \times 60\Omega$)	$60\Omega + 60\Omega + \text{Cap to GND}$	Better EMI filtering, same effective 60Ω load	✓ CORRECT
Too many terminators	3 or more 120Ω	Parallel resistance $< 40\Omega$, excessive current draw	✗ FAIL

8. Physical Layer — High Speed vs Low Speed

Parameter	CAN High Speed (ISO 11898-2)	CAN Low Speed (ISO 11898-3)
Baud Rate	Up to 1 Mbps	Up to 125 kbps
Termination	120Ω at each end	2.2kΩ at each node
Dominant CAN_H	2.75V – 4.5V	1.4V – 4.0V
Dominant CAN_L	0.5V – 2.25V	1.0V – 3.6V
Recessive CAN_H/L	~2.5V both lines	~2.5V both lines (floating)
Differential Voltage (Dom.)	~2V difference	~1.5V difference
Bus Fault Tolerance	Low (single fault kills bus)	High (withstands open/short/ground)
Common Application	Powertrain, OBD, high-speed	Body electronics, fault-tolerant bus



ISO 11898-2 (High Speed CAN) is the standard for modern automotive applications. ISO 11898-3 (Low Speed / Fault Tolerant) is used for body electronics where a single wire short-circuit should not disable all communication.

8.1 Baud Rate vs Cable Length

As baud rate increases, the maximum bus length must decrease. This is because the propagation delay of the signal must stay within the bit time to allow proper arbitration and sampling:

Baud Rate	Max Bus Length	Typical Application
1 Mbps	≤ 40 m	Engine ECU, high-speed in-vehicle network
500 kbps	≤ 100 m	Powertrain CAN, most automotive OBD-II
250 kbps	≤ 250 m	Body control, HVAC, chassis systems
125 kbps	≤ 500 m	Diagnostics, fleet management, J1939
50 kbps	≤ 1000 m	Industrial, off-board diagnostics
10 kbps	≤ 5000 m	Low-speed, long-distance industrial CAN



The bit time at 1 Mbps = 1 microsecond. Signal propagates at ~5 ns/m in copper. At 40m total length (20m round trip), propagation delay ≈ 100ns — safe within one bit time with proper oscillator tolerance.

9. CAN Error Handling & Error Frames

CAN has built-in error detection and handling that operates at the hardware level. This is one of its key advantages over simple UART or SPI protocols.

9.1 Five Error Detection Mechanisms

Error Type	Cause	Detection By	Effect
Bit Error	Transmitted bit $\neq$ monitored bus bit	Transmitter	Immediate retransmit
Stuff Error	>5 consecutive same-polarity bits	All nodes	Error frame sent
CRC Error	Received CRC $\neq$ calculated CRC	Receiver	Error frame sent
Form Error	Fixed-format bit violation	All nodes	Error frame sent
Acknowledgement Error	No dominant ACK from any receiver	Transmitter	Retransmit

9.2 Error Confinement — Node States

CAN uses two counters: Transmit Error Counter (TEC) and Receive Error Counter (REC). Based on these, each node operates in one of three states:

State	TEC/REC Threshold	Behavior
Error Active	TEC < 128 AND REC < 128	Normal operation. Sends active error frames (dominant bits) when error detected.
Error Passive	TEC $\geq$ 128 OR REC $\geq$ 128	Still participates but sends passive error frames (recessive bits). Cannot disturb others.
Bus Off	TEC $\geq$ 256	Node disconnects from bus. Requires 128×11 recessive bits to recover (automatically or manually).

10. CAN FD — Flexible Data Rate

CAN FD (ISO 11898-1:2015) is an extension of Classical CAN that allows higher data-phase baud rates and larger payloads while maintaining backward compatibility with the arbitration phase.

Feature	Classical CAN	CAN FD
Max Baud Rate	1 Mbps	Up to 8 Mbps (data phase)
Max Payload	8 bytes	64 bytes
Two-Speed Operation	No	Yes (arbitration + data phase)
CRC Length	15 bits	17 or 21 bits (enhanced)
Error Detection	Good	Better (longer CRC)
Backward Compatible	—	Arbitration phase at 500kbps / 1Mbps



CAN FD is increasingly used in modern vehicles (AUTOSAR, Ethernet gateway nodes, OTA update networks) where classical CAN bandwidth (< 1 Mbps, max 8 bytes) is insufficient.

11. CAN Arbitration — Who Gets the Bus?

CAN uses Non-Destructive Bitwise Arbitration (also called CSMA/CR — Carrier Sense Multiple Access with Collision Resolution). Every node monitors the bus while transmitting:

- If a node transmits a RECESSIVE bit but reads back a DOMINANT bit, it has lost arbitration
- The losing node immediately stops transmitting and becomes a receiver
- The winning node (lowest CAN-ID = most dominant bits early) continues transmitting
- No data is lost — the winner's frame is intact on the bus
- The losers retry after the bus is free again

Arbitration Example:

Node A transmits ID: 00000000101 (decimal 5)

Node B transmits ID: 00000001001 (decimal 9)

At bit 4: Node B sends recessive (1), but reads back dominant (0) from Node A → Node B LOSES, stops.

Node A (ID=5) wins and completes transmission. Node B retries after.

12. Troubleshooting & Best Practices

12.1 Measuring Termination Resistance

To verify correct bus termination with a multimeter (bus must be POWERED OFF, no ECUs active):

- Set multimeter to resistance (Ω) mode
- Measure resistance between CAN_H and CAN_L
- Expected result with correct termination: 60Ω ($\pm 10\%$)
- If you read $\sim 120\Omega$: only one terminator is present (or the other failed)
- If you read 40Ω or less: too many terminators
- If you read ∞ (open circuit): no termination or broken wire

12.2 Common CAN Problems & Solutions

Problem	Likely Cause	Solution
Bus shows 60Ω but no comms	Incorrect baud rate configuration	Verify all nodes use identical baud rate
Intermittent errors at high speed	Single terminator or stub too long	Add missing terminator; shorten stubs
Node goes Bus-Off repeatedly	Faulty wiring or node TX error	Check wiring; isolate faulty node
All nodes see own messages only	Missing termination $\rightarrow$ no ACK	Add correct termination resistors
CAN_H or CAN_L shorted to GND	Damaged harness or ECU pin short	Inspect wiring, replace damaged section
Frame IDs appear garbled	Mixed Standard/Extended without filter	Configure transceiver/filters correctly
CAN works on bench, fails in car	Longer cables, no stub control	Recalculate termination for actual length

12.3 CAN Design Checklist

- ✓ All nodes on the same bus use the SAME baud rate
- ✓ Exactly TWO 120Ω resistors — one at each physical end
- ✓ No termination resistors at middle nodes or stub ends
- ✓ Bus topology is a straight line, not a star
- ✓ Stub lengths kept within limits for the baud rate used
- ✓ CAN_H and CAN_L are twisted pair (reduces EMI)
- ✓ Shield grounded at ONE point only (to avoid ground loops)
- ✓ All nodes share a common ground reference
- ✓ IDE bit assignment is known (Standard vs Extended) for all messages
- ✓ DBC file is version-controlled and matches firmware implementation

13. Quick Reference Summary

Topic	Key Facts
CAN Standard (2.0A)	11-bit ID, up to 2048 messages, IDE=0
CAN Extended (2.0B)	29-bit ID (11+18), >536M messages, IDE=1
CAN FD	Up to 64 bytes payload, up to 8 Mbps data phase
Dominant bit	Logic 0, wins over recessive
Recessive bit	Logic 1, loses to dominant
Termination rule	EXACTLY $2 \times 120\Omega$ at each END of the bus
Correct bus load	$120\Omega \parallel 120\Omega = 60\Omega$
DBC BO_line	Message ID (decimal), Name, Length (bytes), Sender
DBC SG_formula	Physical = Raw $\times$ Factor + Offset
Baud rate at 1Mbps	Max bus length $\approx 40\text{m}$
Error Active $\rightarrow$ Passive	TEC or REC reaches 128
Bus Off threshold	TEC reaches 256
CAN bus voltages (Dom)	CAN_H $\approx 3.5\text{V}$, CAN_L $\approx 1.5\text{V}$
CAN bus voltages (Rec)	CAN_H = CAN_L $\approx 2.5\text{V}$

— End of CAN Bus Technical Notes —

ISO 11898 | Bosch CAN Specification 2.0 | SAE J1939 | ISO 15765 (UDS/OBD)